

重点回答

· SGLang 的 RadixAttention / RadixCache 解决什么问题？

1. 首先，在没有进行共享前缀的情况下，也就是可能会出现重复存储的情况，在许多轮对话中每个请求都从零开始算KV cache，公共的前缀一直被重复计算，非常浪费时间，效率低下。
2. 对于此方法，是将KV Cache组织成radix tree，然后将用户新的请求中，前缀部分相同的部分进行了公共前缀匹配，命中的部分可以进行复用KV节省算力，未命中部分还是正常通过prefill预填充。
3. 因此解决的是浪费算力和效率低下的问题。

· SGLang 中 page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪一部分？

1. Page-sized KV cache对应的是存储管理部分，在Prefill写入端将算完的KV按照page来分配，同时也是Decode的读取端，Decode的每一步按照page的索引取KV。
2. Prefix reuse对应的是Prefill入口在计算前做的前缀匹配，命中的前缀不需要进行重复计算。

· vLLM 的 PagedAttention 解决什么问题？它和 SGLang 的 RadixAttention / paged KV 机制有什么联系与差异？

1. PagedAttention解决的是传统的LLM架构中，给每个请求都进行了连续的预分配而且都按照**最大长度**预留，会导致显存浪费非常多，在论文中有数据体现。然后用的是分页将KV Cache切成KV block，然后再用block table映射来减少浪费，提升吞吐量。
2. 这个方法与SGLang的机制的分页思想有关联，SGLang内部也用了分页管理KV，二者目标一致，都是为了降低显存的浪费。
3. SGLang主要解决的是时间上的问题，重复前缀匹配主要是时间算力的浪费；而PagedAttention解决的是空间上的，利用KV block让连续预分配导致的空间浪费减少，大大利用空间。